

Universidad Mayor de San Andres
Facultad de Ciencias Puras y Naturales
Carrera de Informática



PROYECTOS INDIVIDUALES

Nombre: Machaca Callisaya Joselin Gladys

Docente: Silva Choque Moises

Sigla: Inf-324

Materia: Programación Multimedia

PROYECTOS INDIVIDUALES

Ejercicio1: Clasificación de texturas

Desarrollar una aplicación capaz de diferenciar superficies dentro de una imagen, por ejemplo césped, tierra, cemento o asfalto, aplicando principios similares a los utilizados en clasificación de imágenes satelitales

Funcionamiento del programa

- Desarrollado en C# utilizando Windows Forms.
- Permite cargar una imagen y mostrarla en pantalla.
- El usuario puede seleccionar una zona haciendo clic sobre la imagen.
- Se analiza una ventana de píxeles alrededor del punto seleccionado.
- Se calculan los valores promedio de los colores RGB (Rojo, Verde y Azul).
- Mediante reglas de clasificación, la textura se identifica como: Césped, Tierra, Cemento, Asfalto, Desconocido

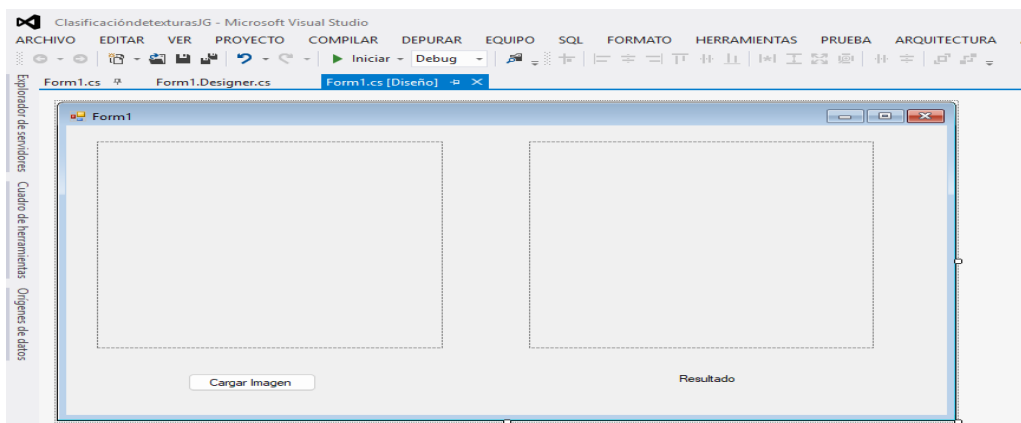
Se muestran:

- Coordenadas del punto seleccionado.
- Valores RGB promedio.
- Clase detectada.
- La región analizada se resalta con un rectángulo rojo.
- Incluye un método de clasificación por ventanas para analizar toda la imagen.
- Genera una nueva imagen donde cada textura se representa mediante un color específico.

Resultado

- Clasificación automática de texturas basada en el análisis de color RGB.
- Identificación visual de diferentes superficies dentro de una imagen.

Diseño



Código

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace ClasificacióndetexturasJG
{
    public partial class Form1 : Form
    {
        Bitmap imagen;
        public Form1()
        {
            InitializeComponent();
        }
        private void pictureBox1_MouseClick(object sender, MouseEventArgs e)
        {
            if (imagen == null)
                return;
            string clase;
            int r, g, b;
            ObtenerInformacionPunto(
                imagen,
                e.X,
                e.Y,
                out clase,
                out r,
                out g,
                out b);
            Bitmap copia = new Bitmap(imagen);
            using (Graphics grafico = Graphics.FromImage(copia))
            {
                grafico.DrawRectangle(
                    Pens.Red,
                    e.X - 10,
                    e.Y - 10,
                    20,
                    20);
            }
            pictureBox2.Image = copia;
            Resultado.Text =
                "X=" + e.X +
                " Y=" + e.Y +
```

```

        " RGB(" + r + "," + g + "," + b + ")" +
        "->" + clase;

    }

    private void cargarImagen_Click(object sender, EventArgs e)
    {
        OpenFileDialog abrir = new OpenFileDialog();
        abrir.Filter = "Imagenes|*.jpg;*.jpeg;*.png;*.bmp";
        if (abrir.ShowDialog() == DialogResult.OK)
        {
            imagen = new Bitmap(abrir.FileName);
            pictureBox1.Image = imagen;
        }
    }

    private Bitmap ClasificarPorVentanas(Bitmap bmp)
    {
        Bitmap salida = new Bitmap(bmp.Width, bmp.Height);
        int radio = 2;
        for (int y = radio; y < bmp.Height - radio; y++)
        {
            for (int x = radio; x < bmp.Width - radio; x++)
            {
                long sumaR = 0;
                long sumaG = 0;
                long sumaB = 0;
                int contador = 0;
                for (int j = -radio; j <= radio; j++)
                {
                    for (int i = -radio; i <= radio; i++)
                    {
                        Color c = bmp.GetPixel(x + i, y + j);
                        sumaR += c.R;
                        sumaG += c.G;
                        sumaB += c.B;
                        contador++;
                    }
                }
                int promR = (int)(sumaR / contador);
                int promG = (int)(sumaG / contador);
                int promB = (int)(sumaB / contador);
                Color colorClase;
                if (promG > promR && promG > promB)
                {
                    colorClase = Color.Lime;
                }
                else if (promR > promG && promR > promB)
                {
                    colorClase = Color.SaddleBrown;
                }
            }
        }
    }

```

```

    }
    else if ((promR + promG + promB) / 3 > 140)
    {
        colorClase = Color.LightGray;
    }
    else
    {
        colorClase = Color.Black;
    }
    salida.SetPixel(x, y, colorClase);
}
}
return salida;
}
private void ObtenerInformacionPunto(
    Bitmap bmp,
    int xCentro,
    int yCentro,
    out string clase,
    out int promR,
    out int promG,
    out int promB)
{
    int radio = 5;
    long sumaR = 0;
    long sumaG = 0;
    long sumaB = 0;
    int contador = 0;
    for (int y = yCentro - radio; y <= yCentro + radio; y++)
    {
        for (int x = xCentro - radio; x <= xCentro + radio; x++)
        {
            if (x >= 0 &&
                x < bmp.Width &&
                y >= 0 &&
                y < bmp.Height)
            {
                Color c = bmp.GetPixel(x, y);
                sumaR += c.R;
                sumaG += c.G;
                sumaB += c.B;
                contador++;
            }
        }
    }
    promR = (int)(sumaR / contador);
    promG = (int)(sumaG / contador);
    promB = (int)(sumaB / contador);
}

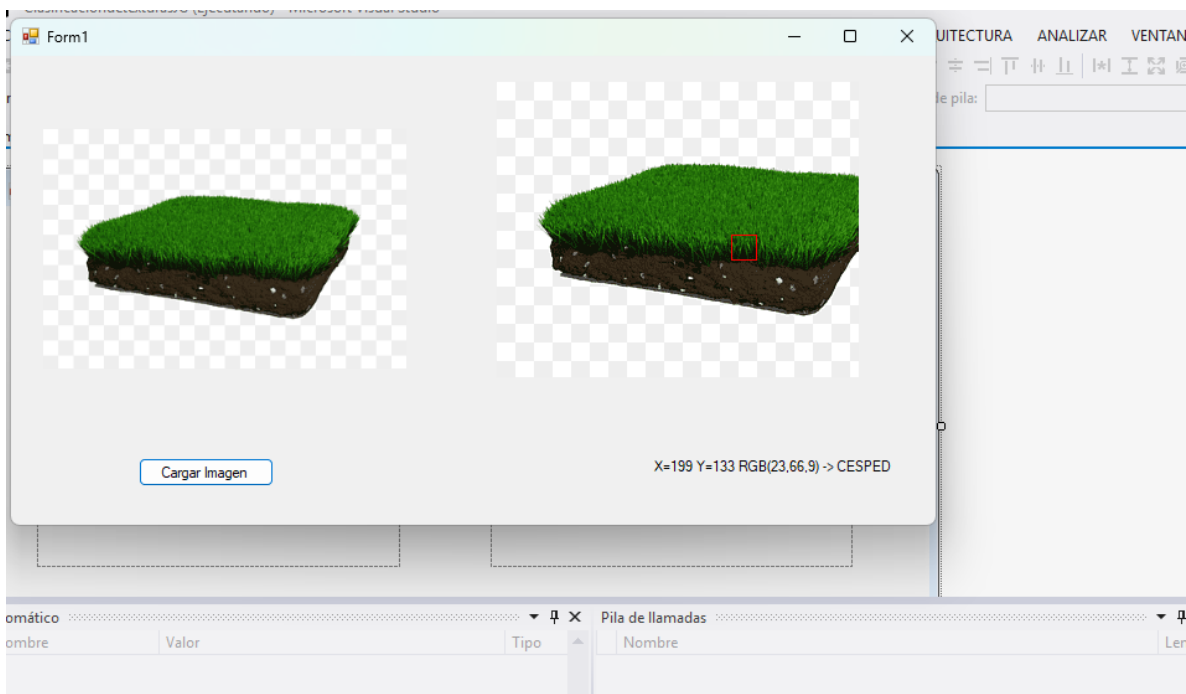
```

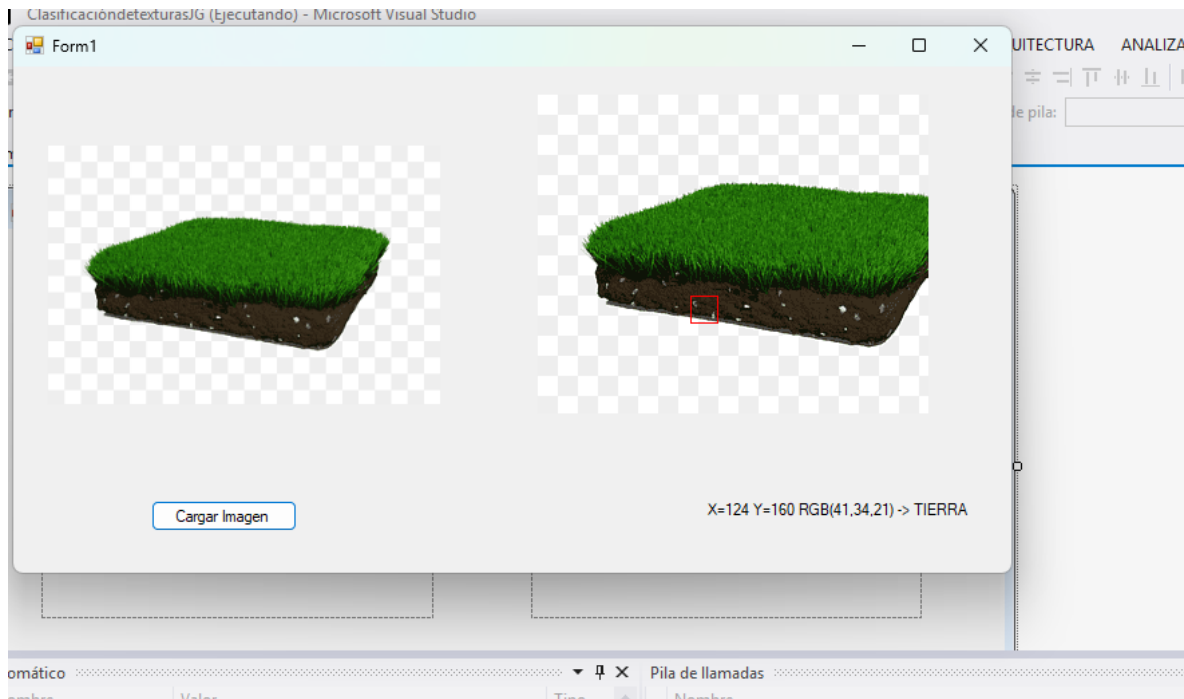
```

if (promG > promR + 25 && promG > promB + 25)
{
    clase = "CESPED";
}
else if (promR > promB + 10)
{
    clase = "TIERRA";
}
else if (Math.Abs(promR - promG) < 15 && Math.Abs(promG - promB) < 15)
{
    int gris = (promR + promG + promB) / 3;
    if (gris >= 150 && gris <= 210)
        clase = "CEMENTO";
    else if (gris >= 80 && gris <= 130)
        clase = "ASFALTO";
    else
        clase = "DESCONOCIDO";
}
else
{
    clase = "DESCONOCIDO";
}
}
}
}
}

```

Ejecución





Ejercicio 2: Implementación de un filtro suavizado

Diseñar un filtro de promedio que recorra la imagen utilizando ventanas de 3x3 píxeles para reducir ruido y suavizar transiciones de color.

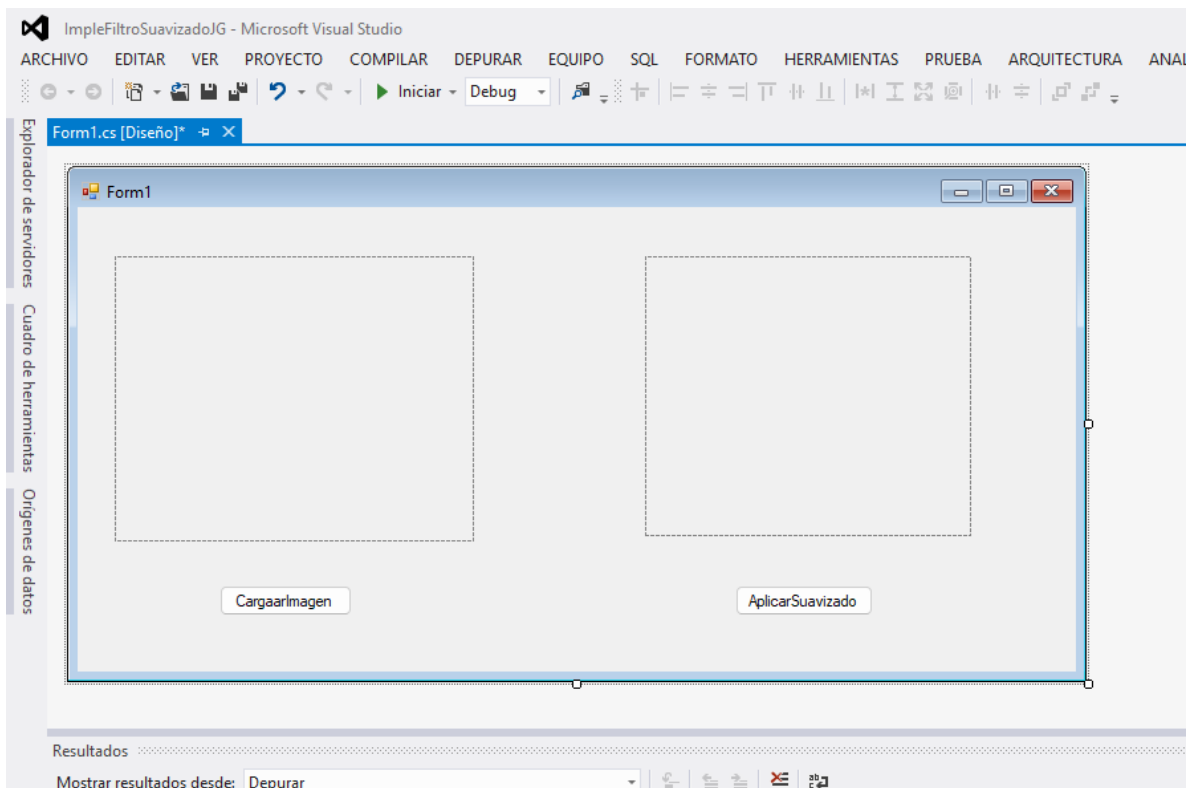
Funcionamiento del programa

- Desarrollado en **C# utilizando Windows Forms**.
- Permite **cargar una imagen** y visualizarla en pantalla.
- Aplica un **filtro de suavizado** para reducir ruido y variaciones bruscas de color.
- Utiliza una **ventana de vecindad de 3x3 píxeles** alrededor de cada píxel.
- Calcula el promedio de los valores **RGB (Rojo, Verde y Azul)** de los píxeles vecinos.
- Asigna el color promedio al píxel central para obtener una imagen más uniforme.
- Procesa toda la imagen píxel por píxel.
- Conserva los bordes originales mediante un método auxiliar que copia los píxeles exteriores.
- Muestra la imagen original y la imagen suavizada para facilitar la comparación.

Resultado

- Reducción del ruido presente en la imagen.
- Suavización de cambios bruscos de color.
- Obtención de una imagen más uniforme y preparada para posteriores procesos de análisis o procesamiento digital.

Diseño



Código

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace ImpleFiltroSuavizadoJG
{
    public partial class Form1 : Form
    {
        Bitmap imagen;
        Bitmap imagenProcesada;
        public Form1()
        {
            InitializeComponent();
        }
        private void CargarImagen_Click(object sender, EventArgs e)
        {
            OpenFileDialog abrir = new OpenFileDialog();
```

```

        abrir.Filter = "Imagenes|*.jpg;*.jpeg;*.png;*.bmp";
        if (abrir.ShowDialog() == DialogResult.OK)
        {
            imagen = new Bitmap(abrir.FileName);
            pictureBox1.Image = imagen;
        }
    }

    private void AplicarSuavizado_Click(object sender, EventArgs e)
    {
        if (imagen == null)
        {
            MessageBox.Show("Primero debes cargar una imagen original.", "Aviso",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        this.Cursor = Cursors.WaitCursor;
        Application.DoEvents();
        int ancho = imagen.Width;
        int alto = imagen.Height;
        imagenProcesada = new Bitmap(ancho, alto);
        for (int x = 1; x < ancho - 1; x++)
        {
            for (int y = 1; y < alto - 1; y++)
            {
                int acumuladorR = 0;
                int acumuladorG = 0;
                int acumuladorB = 0;

                for (int i = -1; i <= 1; i++)
                {
                    for (int j = -1; j <= 1; j++)
                    {
                        Color pixelVecino = imagen.GetPixel(x + i, y + j);
                        acumuladorR += pixelVecino.R;
                        acumuladorG += pixelVecino.G;
                        acumuladorB += pixelVecino.B;
                    }
                }

                int nuevoR = acumuladorR / 9;
                int nuevoG = acumuladorG / 9;
                int nuevoB = acumuladorB / 9;

                imagenProcesada.SetPixel(x, y, Color.FromArgb(nuevoR, nuevoG, nuevoB));
            }
        }
    }

```

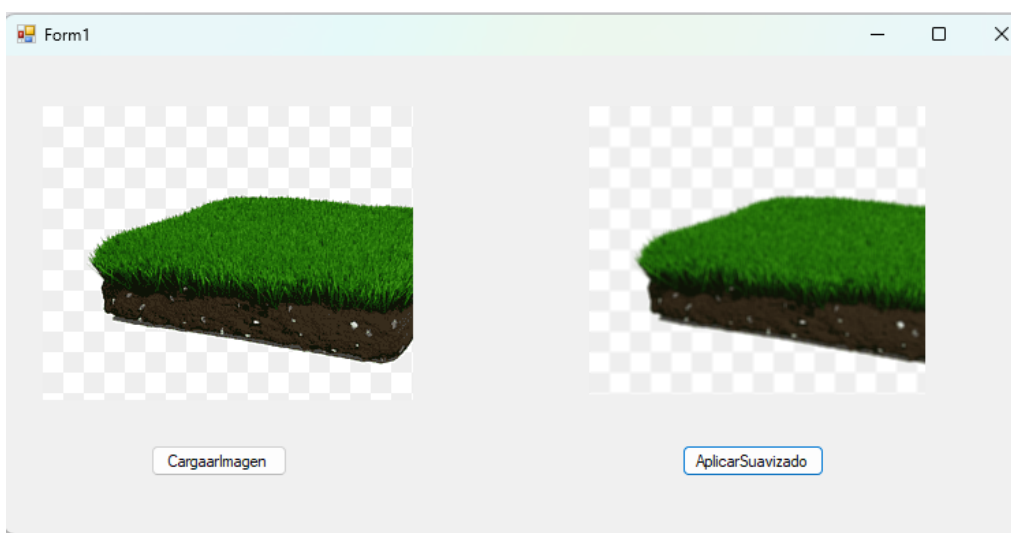
```

        ClonarBordes(imagen, imagenProcesada);

        pictureBox2.Image = imagenProcesada;
        this.Cursor = Cursors.Default;
        // Notificación flotante de finalización
        //MessageBox.Show("Filtro aplicado con éxito.\nTiempo de ejecución:
{sw.ElapsedMilliseconds} ms.", "Proceso Terminado", MessageBoxButtons.OK,
MessageBoxIcon.Information);
    }
    private void ClonarBordes(Bitmap origen, Bitmap destino)
    {
        int ancho = origen.Width;
        int alto = origen.Height;
        // Columnas laterales (Izquierda y Derecha)
        for (int y = 0; y < alto; y++)
        {
            destino.SetPixel(0, y, origen.GetPixel(0, y));
            destino.SetPixel(ancho - 1, y, origen.GetPixel(ancho - 1, y));
        }
        // Filas horizontales (Arriba y Abajo)
        for (int x = 0; x < ancho; x++)
        {
            destino.SetPixel(x, 0, origen.GetPixel(x, 0));
            destino.SetPixel(x, alto - 1, origen.GetPixel(x, alto - 1));
        }
    }
}

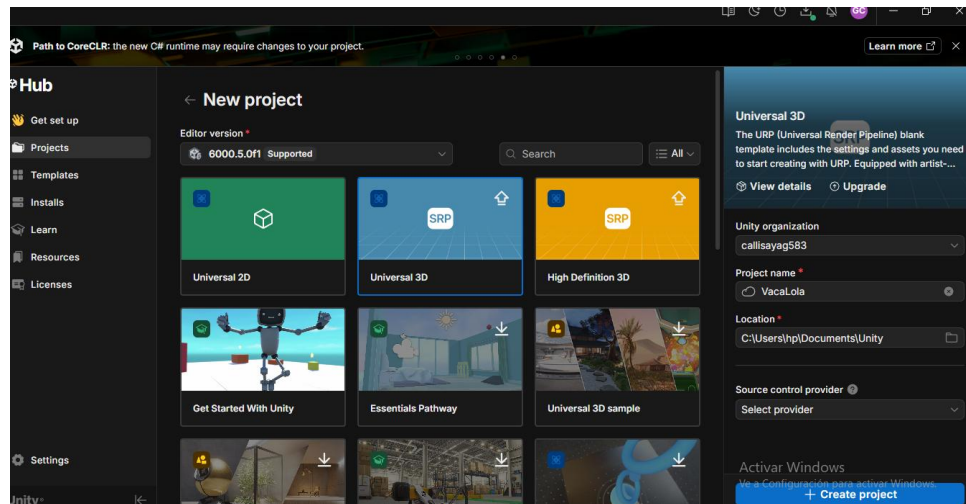
```

Ejecución

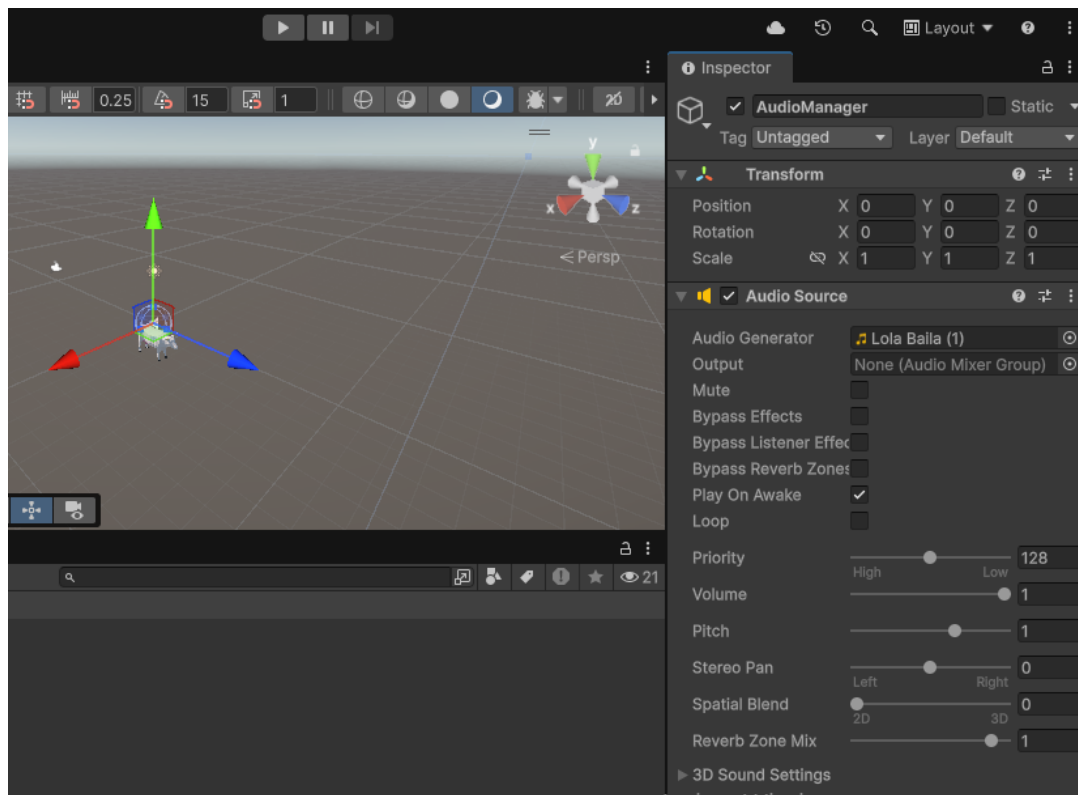


Ejercicio 3: Cover de la “vaca lola”

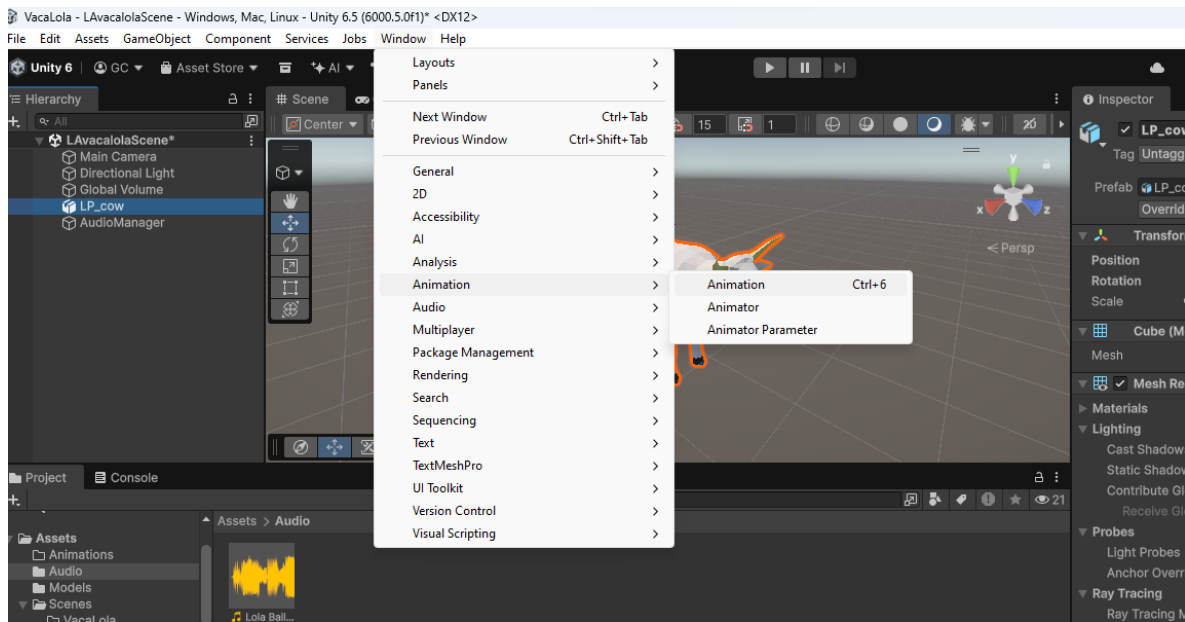
Abrimos Unity hub y creamos el proyecto de vacaOf, previamente debe tener un audio mp3.



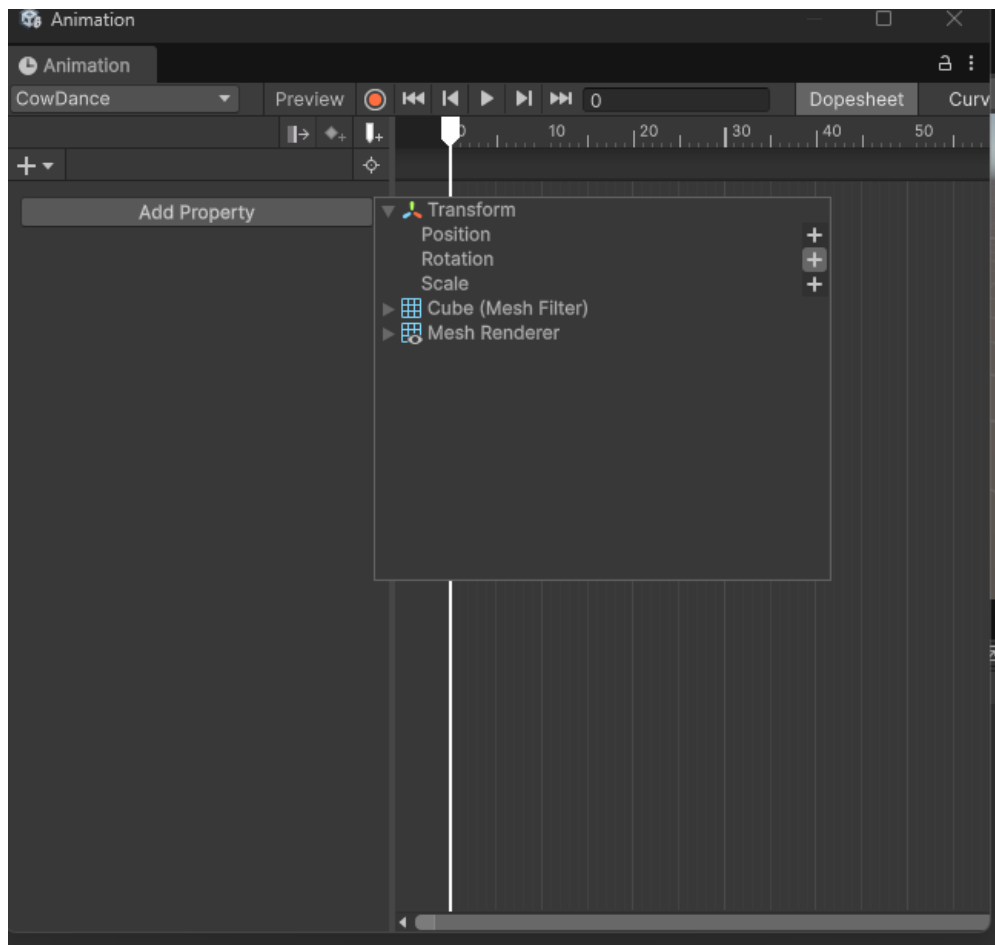
Una vez dentro del proyecto creado, importamos el audio seleccionado en Audio Source y la imagen en 3D.



Para la animación: Ingresamos a las pestañas como se ve en la imagen



Una vez dentro, se procede a mover el objeto e ir capturando los movimientos que se quiera obtener, mediante la posición y/o rotación.



Finalmente ejecutar con el en botón play.